SQL & MySQL

A Complete Beginner-to-Intermediate Guide

Databases, SQL Basics, Joins, Functions, Keys, Normalization, Window Functions

Compiled, corrected, and explained from personal study notes (Edition 2)

1. Introduction to Databases

1.1 Data vs Information

Before understanding databases, it is important to understand the difference between raw data and information.

Term	Meaning	Example
Raw Data	Facts and figures with no context on their own.	"Krish 500"
Information	Data that has been processed or given context so it becomes meaningful.	"Krish earns 5000"

Without a database, it becomes very difficult to efficiently perform CRUD operations (Create, Read, Update, Delete) on data. This is exactly the problem a database solves.

1.2 What is a Database (DB)?

A database is basically a container — an organized system used to store, manage, and retrieve data efficiently and reliably. Instead of scattering data across files, a database keeps it structured so it can be searched, updated, and secured easily.

1.3 Types of Databases (Structural vs Non-Structural)

Structural (Structured)	Non-Structural (Unstructured / Semi-structured)
Data is organized in a fixed, predefined format (rows and columns), e.g. relational databases.	Data does not follow a fixed format — e.g. documents, JSON, images, videos, key-value pairs used by NoSQL databases.

1.4 Ways to Classify Databases

- Based on the Data Model — how the data is structurally organized (RDBMS, NoSQL, Hierarchical, Object-Oriented, Network DB).
- Based on Usage / Development — the purpose the database serves in an application (transactional systems, analytical systems, etc.).

2. Types of Databases Based on Data Model

Type	How Data is Stored	Examples	Typical Use Case
RDBMS	Rows & Columns (tables)	MySQL, Oracle	Banking, ERP systems

Type	How Data is Stored	Examples	Typical Use Case
NoSQL	Documents, key-value pairs, column families	MongoDB, Redis	Social media, Big Data
Hierarchical DB	Tree structure (parent-child)	IBM IMS, XML	Old mainframe systems
Object-Oriented DB	Objects, similar to OOP concepts	db4o	Gaming, multimedia, CAD
Network DB	Graph-like structure with multiple links	IDMS	Old telecom systems

2.1 RDBMS — Relational Database Management System

RDBMS stands for Relational Database Management System. Its structure is based on tables that are linked together using Primary Keys and Foreign Key relationships.

- SQL (Structured Query Language) — the standard language used to communicate with an RDBMS.
- NoSQL ("Not Only SQL") — used when data is huge in volume and unstructured in nature. It does not strictly require a fixed schema.

2.2 Why is RDBMS often preferred?

- Less data duplication because of normalization.
- High data integrity, since relationships and constraints are enforced.
- Higher security controls.
- Supports a large number of concurrent users reliably.

2.3 Relational DB vs Non-Relational DB

Feature	Relational Database	Non-Relational (NoSQL) Database
Structure	Structured — organized into tables	Semi-structured / unstructured
Example	MySQL	MongoDB
Query Language	SQL	NoSQL, JSON-based, CQL, etc.
Joins	Supported	Not supported (generally)
Schema	Pre-defined (fixed)	Dynamic and flexible
Scaling	Vertical scaling — needs bigger servers	Horizontal scaling — add more servers
Transactions	ACID compliant	BASE (Basically Available, Soft state, Eventually consistent)
Data Integrity	High — strong constraint support	Lower — constraints often not enforced

Feature	Relational Database	Non-Relational (NoSQL) Database
Speed	Great for complex queries with relationships	Very fast for simple reads & writes

Scaling: Vertical vs Horizontal

- Vertical Scaling — increasing the capacity of a single server (more RAM, CPU, storage).
- Horizontal Scaling — adding more servers/machines to share the load.

2.4 NoSQL Sub-types (Based on Structure)

Sub-type	Description	Example
Document DB	Stores data in JSON-like documents (key-value pairs inside a document).	MongoDB
Key-Value DB	Works like a dictionary — every value is accessed through a unique key.	Redis
Column DB	Stores data column-wise instead of row-wise; columns are stored together.	Cassandra
Graph DB	Stores data as nodes and relationships (edges), ideal for connected data.	Neo4j

2.5 DBMS vs RDBMS

DBMS	RDBMS
No normalization enforced	Follows normalization rules
No relationships between data	Relationships via Primary Key & Foreign Key
Example: XML file-based systems	Example: MySQL, PostgreSQL

> ***Note:*** *DBMS is the general umbrella term for any software that manages databases. RDBMS is a specific type of DBMS that stores data in structured tables and enforces relationships.*

3. Data Types in SQL

A data type defines the kind of value that can be stored in a particular column of a table. Choosing the right data type keeps data accurate and storage efficient.

- Numeric Data Types
- String Data Types
- Date & Time Data Types
- Boolean Data Type

3.1 Numeric Data Types

Data Type	Description
INT	Stores whole numbers (integers). Takes up 4 bytes of storage.
TINYINT	A smaller integer type, takes up only 1 byte.
DECIMAL(M, D)	Stores exact fixed-point numbers. M = total digits, D = digits after the decimal point. Example: DECIMAL(5,2) can store values up to 999.99.

3.2 String Data Types

Data Type	Description
VARCHAR(size)	Stores variable-length text; memory is allocated based on the actual length entered, up to the defined size.
CHAR(size)	Stores fixed-length text; memory is always allocated for the full defined size, regardless of actual text length.
TEXT / BLOB	Used to store large amounts of text (Binary Large Object). TEXT can typically store up to about 65,000 characters.

> ***Note:*** *Use CHAR for fields with a consistent length (like a 2-letter country code) and VARCHAR for fields whose length varies (like a name or an email address).*

3.3 Date & Time Data Types

Data Type	Format	Description
DATE	YYYY-MM-DD	Stores only the date.
DATETIME	YYYY-MM-DD HH:MM:SS	Stores both date and time.
TIMESTAMP	YYYY-MM-DD HH:MM:SS	Stores date and time, and can automatically update itself when the row is modified.

3.4 Boolean Data Type

Used to store logical values — true or false (in MySQL this is often internally represented as TINYINT(1), where 1 = true and 0 = false).

4. Choosing a Database: MySQL vs Oracle

Criteria	MySQL	Oracle
Cost	Free (open-source)	Paid (licensed)
Flexibility	Flexible and easy to work with	Feature-rich but more rigid setup
Ease of Use	Beginner-friendly	Steeper learning curve
Usage Limits	Fewer restrictions for common use	More enterprise-grade limits/controls

> ***Note:*** *This is a simplified comparison. In practice the right choice depends on scale, budget, and the specific features an organization needs — both are powerful, production-grade RDBMS platforms.*

5. Constraints

Constraints are a set of rules applied to columns in a table to validate and control what data can be entered. They help maintain the accuracy and reliability of data inside the database.

Constraint	Purpose
NOT NULL	Ensures a column cannot have a NULL (empty) value.
UNIQUE	Ensures all values in a column are different from one another. A column with only a UNIQUE constraint can still accept one NULL value.
PRIMARY KEY	A special constraint that is both UNIQUE and NOT NULL. It uniquely identifies each row in a table. A table can have only one primary key.
FOREIGN KEY	Used to establish a relationship between two tables. A table can have multiple foreign keys, each referencing a primary key in another table.
CHECK	Checks that values meet a specific condition before being inserted. Multiple conditions can be combined into a single CHECK constraint.
DEFAULT	Automatically assigns a default value to a column when no value is provided during insertion.

```
-- Example: constraints in a CREATE TABLE statement
CREATE TABLE emp (
    empid    INT PRIMARY KEY,
    ename    VARCHAR(50) NOT NULL,
    email    VARCHAR(100) UNIQUE,
    sal      DECIMAL(8,2) DEFAULT 0,
    deptno   INT,
    CHECK (sal >= 0),
    FOREIGN KEY (deptno) REFERENCES department(deptno)
);
```

6. Keys in a Database

A key is one or more columns used to uniquely identify rows in a table, or to build relationships between tables. Understanding keys properly also lays the foundation for Normalization, covered later in this document.

6.1 Types of Keys

Key	Description
Candidate Key	Any column (or set of columns) that qualifies to uniquely identify each row in a table. A table can have multiple candidate keys.
Primary Key	The candidate key that is actually chosen to uniquely identify rows in the table. It must be UNIQUE and NOT NULL, and a table can only have one.

Key	Description
Alternate Key	All the candidate keys that were NOT chosen as the primary key. They could have worked as the primary key but were not selected.
Super Key	A set of one or more attributes that can uniquely identify a row — it may include extra columns beyond what is strictly needed. Every candidate key is a super key, but not every super key is a candidate key (a candidate key is the smallest/minimal super key).
Foreign Key	An attribute in one table that refers to the primary key of another table, used to establish a relationship between the two tables.
Composite Key	A key made up of two or more columns combined together, where no single column alone can uniquely identify a row.

6.2 Key Attribute vs Non-Key Attribute

- Key Attribute (also called Prime Attribute) — a column that is part of any candidate key. It is expected to hold unique values.
- Non-Key Attribute (also called Non-Prime Attribute) — any column in the table that is not part of a candidate key.

Example — Employee table:

```
employee(empno, ename, sal, job, hiredate, deptno, email, phone)

Key attributes (part of some candidate key): empno, email, phone
Non-key attributes:                          ename, sal, job, hiredate, deptno

Primary key         -> empno
Alternate/candidate -> email, phone
Super key example   -> (empno, phone, email)   -- a valid, if redundant, super key
```

6.3 AUTO_INCREMENT

AUTO_INCREMENT is used so that a numeric column (typically the primary key) automatically generates the next sequential value every time a new row is inserted, without the value needing to be supplied manually.

Example: "Create a table called login with login_id, user_name, login_time, and update_time."

```
CREATE TABLE login (
    login_id     INT AUTO_INCREMENT PRIMARY KEY,
    user_name    VARCHAR(50) NOT NULL,
    login_time   DATETIME,
    update_time  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

7. ER Diagram / ER Model

The Entity Relationship (ER) Model is a conceptual blueprint of a database. It represents entities, their attributes, and the relationships (connections) between them. The ER model is used before schema design to bring clarity to how the data will be organized.

7.1 Key Components

Component	Meaning	Represented As
Entity	Anything from the real world that data is being stored about (e.g. Student, Employee, Department, Course).	Rectangle
Attribute	A property or characteristic of an entity (e.g. empid, ename, job).	Oval
Relationship	An association between two or more entities (e.g. "Students choose Courses").	Diamond

7.2 Cardinality

Cardinality describes how many instances of one entity relate to instances of another entity.

- 1 : 1 — One instance relates to exactly one instance of another entity.
- 1 : N — One instance relates to many instances of another entity.
- N : N (Many-to-Many) — Requires an intermediate (junction) table to be created to manage the relationship.

> ***Note:*** *A foreign key can be taken from any of the related tables to represent a 1:1 or 1:N relationship, but for N:N relationships, an intermediate/junction table is required.*

7.3 Example: Employee – Department

Entities: Employee and Department (related in a 1 : M relationship — one department has many employees).

- Employee attributes: empid, ename, job
- Department attributes: deptno, dname, loc

8. Database Schema

A schema is the logical container or blueprint for database objects. It defines the tables, columns, data types, and constraints — essentially acting as a map of how data is stored and accessed.

8.1 Why do we need a Schema?

- To avoid redundancy (duplicate/repeated data).

- To support normalization.
- To give a clear overview of the entire database structure.

8.2 Components of a Schema

- Tables — store the data using rows and columns.
- Columns — define the attributes, along with their data type and constraints.
- Rows — the individual records; constraints are the rules applied on them.

8.3 ER Model vs Schema

ER Model	Schema
Conceptual design	Physical design
Shows entities, attributes, and relationships	Defines the actual tables, columns, and constraints
Used for planning	Used for implementation

9. Types of SQL Statements

SQL statements are grouped into five categories, based on the kind of task they perform:

#	Category	Full Form	Purpose
1	DDL	Data Definition Language	Defines and modifies the structure of database objects.
2	DML	Data Manipulation Language	Manipulates the actual data stored inside tables.
3	TCL	Transaction Control Language	Manages transactions within the database.
4	DCL	Data Control Language	Controls access/permissions to the database.
5	DQL	Data Query Language	Used to fetch/retrieve data from the database.

9.1 DDL — Data Definition Language

Commands: CREATE, ALTER, RENAME, DROP, TRUNCATE

Command	Description
CREATE	Creates a new database or table.
ALTER	Modifies the structure of an existing database/table.
DROP	Permanently deletes a table or database, including its structure.
TRUNCATE	Removes all rows from a table in one shot, but keeps the table structure.
RENAME	Renames a database object (e.g. a table).

DROP vs TRUNCATE vs DELETE

- DELETE removes rows one record at a time, which makes it slower — this is why it is not recommended for clearing large tables.
- TRUNCATE deletes all records in one shot (much faster) but keeps the table structure intact. It cannot be used with a WHERE clause.
- DROP removes the table (or database) permanently, structure included — it no longer exists afterward.

9.2 DML — Data Manipulation Language

Commands: INSERT, UPDATE, DELETE

Command	Description
INSERT	Adds new rows of data into a table.
UPDATE	Modifies existing data within a table.

Command	Description
DELETE	Removes specific rows from a table, based on a condition.

9.3 TCL — Transaction Control Language

Commands: COMMIT, ROLLBACK, SAVEPOINT

Command	Description
COMMIT	Saves all the changes made during the current transaction permanently.
ROLLBACK	Undoes changes made during the current transaction process.
SAVEPOINT	Sets a point within a transaction to which you can roll back later, if needed.

9.4 DCL — Data Control Language

Commands: GRANT, REVOKE

Command	Description
GRANT	Gives specific access permissions to a database user.
REVOKE	Takes away previously granted permissions from a user.

9.5 DQL — Data Query Language

The DQL command is SELECT — used to fetch and retrieve data.

Term	Description
SELECT	Selects data and displays it on the screen.
Projection	Used to get specific column(s) — i.e. controlling which columns are shown.
Selection	Used to filter which rows are returned, based on a condition (can involve both rows and columns).
Join	Used to combine and fetch data from two (or more) tables simultaneously.

9.6 CRUD Operations

CRUD is a common acronym summarizing the four basic operations performed on data — these map directly onto DDL/DML commands:

Letter	Operation	Corresponding SQL
C	Create / Insert	INSERT (DML), CREATE (DDL)

Letter	Operation	Corresponding SQL
R	Read / Retrieve	SELECT (DQL)
U	Update / Modify	UPDATE (DML)
D	Delete	DELETE (DML), DROP (DDL)

10. The SELECT Statement

The general syntax of a SELECT query is:

```
SELECT * | DISTINCT column_name | expression [alias]
FROM table_name
WHERE <filter_conditions>;
```

- The WHERE clause is used to filter records based on a condition.
- SQL values/keywords are not case-sensitive — queries can be written in any case.
- A clause is a specific keyword that performs a specific task within a query (e.g. WHERE, FROM, GROUP BY).

10.1 Projections

Projection refers to selecting specific columns from a table. The asterisk (*) can be used alone, or as emp.* — both work the same way and return all columns.

```
SELECT * FROM emp;
```

DISTINCT

DISTINCT is used to return only the unique values/records from the given column(s).

```
-- Returns each unique salary value
SELECT DISTINCT sal FROM emp;

-- Returns unique combinations (paired) of salary and job
SELECT DISTINCT sal, job FROM emp;
```

Expressions and Aliases

An expression combines operands with an operator — for example, sal + 200. An alias (AS) is used to give a temporary, more readable name to a column or expression in the output.

```
SELECT sal + 200 AS bonus_sal FROM emp;

-- AS is optional
SELECT sal + 200 bonus_sal FROM emp;
```

10.2 Selections (Filtering with WHERE)

Example: "Display names of employees who were hired after 1981."

```
SELECT ename FROM emp
WHERE hiredate > '1981-12-31';
```

```
-- Alternative using YEAR() function
SELECT ename FROM emp
WHERE YEAR(hiredate) > 1981;
```

Example: "Display name and job of employees who are not a clerk."

```
SELECT ename, job FROM emp WHERE job = 'CLERK';   -- equals
SELECT ename, job FROM emp WHERE job <> 'CLERK';  -- not equal to
```

> ***Note:*** *Multiple conditions can be combined inside a WHERE clause using logical operators such as AND, OR, and NOT.*

11. Operators in SQL

Operators are symbols that perform specific operations on values or expressions.

Category	Examples
Arithmetic Operators	+, -, *, /, %
Relational / Comparison Operators	=, <>, >, <, >=, <=
Concatenation Operator	|| (or CONCAT() in MySQL)
Logical Operators	AND, OR, NOT
Special Operators	IN, NOT IN, BETWEEN, NOT BETWEEN, IS, IS NOT, LIKE, NOT LIKE
Subquery Operators	ALL, ANY, EXISTS, NOT EXISTS

11.1 Special Operators

IN / NOT IN

Used to check whether a value matches any value within a given list.

```
SELECT name FROM emp WHERE ename IN ('SMITH', 'MOHAN');
```

BETWEEN

Used for a range of values. The BETWEEN operator is inclusive of both boundary values, and the range order cannot be swapped (the smaller value must come first).

```
SELECT * FROM emp WHERE sal BETWEEN 3000 AND 6000;
```

IS / IS NOT (NULL checks)

The equality operator (=) cannot be used to compare with NULL, because NULL represents an unknown value, not a comparable one. The IS operator must be used along with the NULL keyword to check for NULL values. IS NOT is used to reject/exclude NULL values.

Example: "Display names of employees who do not have any reporting manager."

```
SELECT ename FROM emp WHERE mgr IS NULL;
```

Example: "Display name, job, and salary of employees who have a reporting manager and belong to department 30."

```
SELECT ename, job, sal FROM emp WHERE mgr IS NOT NULL AND deptno = 30;
```

LIKE / NOT LIKE (Pattern Matching)

LIKE is used for pattern-based searches on string values, using wildcard characters: % (any number of characters) and _ (exactly one character).

Pattern	Meaning
'%R%R%'	Contains the letter R at least twice
'%LL%'	Contains two consecutive L's
'S%T'	Starts with S and ends with T
'_A%'	Has 'A' as the second character
'%E_'	Has 'E' as the second-to-last character
'____'	Exactly 4 characters long (four underscores)

11.2 Subquery Operators

ANY

The ANY operator selects records that are true for any one of the values returned by a subquery. It works similarly to the OR operator. Common forms: > ANY, < ANY, >= ANY, <= ANY.

ALL

The ALL operator selects records that are true for every value returned by the subquery — it works similarly to the AND operator. Common forms: > ALL, < ALL, >= ALL, <= ALL.

11.3 Combining Multiple Conditions

Example: "Display names, job, and deptno of employees who work as CLERK and belong to either department 10 or 30."

```
SELECT ename, job, deptno FROM emp
WHERE job = 'CLERK' AND (deptno = 10 OR deptno = 30);
```

12. Joins

A JOIN is used to combine and retrieve related data from two (or more) tables simultaneously, based on a common column between them. Joins are one of the most important concepts in relational databases, since normalized data is spread across multiple tables.

12.1 Types of Joins

- Cross Join
- Natural Join / Inner Join
- Outer Join
    - a. Left Outer Join
    - b. Right Outer Join
    - c. Full Outer Join
- Self Join

12.2 Join Condition

A join condition tells SQL which rows from each table should be matched together. It is mandatory for Inner, Outer, and Self joins (Cross Join is the only one that does not use a join condition). The condition is written using a common column shared by both tables:

```
table1.common_col = table2.common_col

-- Example:
emp.deptno = dept.deptno
```

12.3 Cross Join

In a Cross Join, every record from one table is combined with every record from the other table (this produces the Cartesian Product of the two tables — if table 1 has M rows and table 2 has N rows, the result has M x N rows). No join condition (ON clause) is used.

```
SELECT * | column_name
FROM table1 CROSS JOIN table2;
```

> ***Note:*** *Cross joins are useful in scenarios involving probability, permutations, and combinations — where every possible pairing is genuinely needed. If a WHERE condition is added and refers to a column name that exists in both tables without qualifying it with the table name, MySQL will raise an "ambiguous column" error — the column must be qualified, e.g. table1.column.*

12.4 Inner Join

An Inner Join is used to retrieve only the matching records between two different tables, based on the join condition. If there is no relationship (no matching values) between the two tables, an inner join simply returns no rows for those non-matching records. The columns used to join should ideally share the same data type.

```
SELECT * | col_name
FROM table1 INNER JOIN table2
ON <join_condition>;
```

Example: "Display details of the employees and department table, showing only the matching records."

```
SELECT *
FROM emp INNER JOIN dept
ON emp.deptno = dept.deptno;
```

12.5 Natural Join

A Natural Join automatically joins two tables using all columns that share the same name and data type in both tables — there is no need to write an explicit ON condition.

- It behaves like an INNER JOIN when a genuine relationship (a common column) exists between the tables.
- It behaves like a CROSS JOIN (returning the Cartesian product) when there is no common column between the tables.

```
SELECT * | col
FROM table1 NATURAL JOIN table2;
```

> ***Note:*** *Because Natural Join relies on matching column names automatically, it is used less often in real-world queries — it can silently join on the wrong column if two tables happen to share a common column name by coincidence. An explicit INNER JOIN with ON is generally safer and clearer.*

12.6 Outer Join

An Outer Join is used to retrieve matching records, as well as unmatched records, from one or both of the tables — unlike an Inner Join, which returns only the matches. Where a match doesn't exist, the missing side is filled in with NULL values.

a. Left Outer Join

Returns all the records from the left table, plus only the matching records from the right table. Rows from the left table that have no match in the right table still appear, with NULLs in the right table's columns.

```
SELECT * | col_name
FROM table1 LEFT OUTER JOIN table2
ON table1.common_col = table2.common_col;
```

Example: "Display details of the employee and dept tables, including those employees who are not assigned to any department."

```
SELECT *
FROM emp LEFT OUTER JOIN dept
ON emp.deptno = dept.deptno;
```

b. Right Outer Join

Returns all the records from the right table, plus only the matching records from the left table.

```
SELECT * | col_name
FROM emp RIGHT OUTER JOIN dept
ON emp.deptno = dept.deptno;
```

Example: "Display details of employee and department, including those departments that currently have no employees working in them."

c. Full Outer Join

Returns matching records as well as unmatched records from both tables. MySQL does not have a direct FULL OUTER JOIN keyword, so it is simulated by combining a LEFT JOIN and a RIGHT JOIN with UNION.

```
SELECT * FROM table1 LEFT OUTER JOIN table2
ON table1.common_col = table2.common_col
UNION
SELECT * FROM table1 RIGHT OUTER JOIN table2
ON table1.common_col = table2.common_col;
```

> ***Note:*** *UNION automatically removes duplicate rows, which is exactly why it is used here — otherwise the genuinely matching rows would be counted twice (once from each side).*

12.7 Self Join

A Self Join is used to join a table with itself, in order to compare or retrieve rows of the same table against each other — commonly used for hierarchical data, such as an employee-manager relationship where both the employee and their manager exist as rows in the same emp table.

```
SELECT * | col_name
FROM table1 t1 INNER JOIN table1 t2
ON <self_join_condition>;
```

The self-join condition is written between two aliases of the same table: t1.common_col = t2.common_col, where t1 and t2 are simply two different aliases referring to the same table.

Example: "Display names of employees along with the names of their respective managers."

```
SELECT e.ename, m.ename AS manager_name
FROM emp e INNER JOIN emp m
ON e.mgr = m.empno;
```

13. Functions in SQL

A function is a block of code / a set of instructions that performs a specific task and returns a result.

13.1 Types of Functions

- Predefined (Built-in) Functions
  - - Single-row functions — take input(s) and return one output per row (String, Numeric, Date & Time, Null, Conditional functions).
  - - Multi-row (Aggregate/Group) functions — take multiple input rows and return a single, combined output.
- User-Defined Functions — custom functions created by the user for reusable logic.

13.2 String / Character Functions

Function	Description
UPPER(str)	Converts all characters in a string to uppercase.
LOWER(str)	Converts all characters in a string to lowercase.
CONCAT(str1, str2, ...)	Joins two or more strings together into one.
LENGTH(str) / CHAR_LENGTH(str)	Returns the length of a string (number of characters).
LOCATE(substr, str)	Returns the position of the first occurrence of a character/substring within a string.
SUBSTRING(str, pos, len)	Extracts a substring from a string, starting at position pos, for len characters. A positive position counts from the start (forward); a negative position counts from the end (backward).
REPLACE(str, from_str, to_str)	Replaces every occurrence of from_str with to_str, inside the given string. It is case-sensitive.
TRIM(str)	Removes leading and trailing spaces from a string.

Example: "Convert the lowercase letters of the string 'Malayalam' to uppercase."

```
SELECT UPPER('Malayalam');
-- dual is a dummy table used with SELECT when no real table is needed
SELECT UPPER('Malayalam') FROM dual;
```

Example: "Display every employee's job title in lowercase."

```
SELECT LOWER(job) FROM emp;
```

Example: "Greet every employee with their name and job, e.g. 'Good morning SMITH, you are working as CLERK'."

```
SELECT CONCAT('Good morning ', ename, ', you are working as ', job)
FROM emp;
```

Example: "Display name and salary of employees whose name has exactly 6 characters."

```
SELECT ename, sal FROM emp
WHERE LENGTH(ename) = 6;
```

Example: "Find the position of the character 'A' in the string 'SUARBH'."

```
SELECT LOCATE('A', 'SUARBH');
```

Example: "Find employees whose name has 'A' as its second character."

```
SELECT * FROM emp
WHERE LOCATE('A', ename) = 2;
```

Example: "Replace every occurrence of the letters 'MAN' in the job column with 'WOMAN'."

```
SELECT REPLACE(job, 'MAN', 'WOMAN') FROM emp;
```

13.3 Numeric Functions

Function	Description
ROUND(num, dec)	Rounds a number to the given number of decimal places.
TRUNCATE(num, dec)	Truncates (cuts off) a number to the given number of decimal places, without rounding.
MOD(num, divisor)	Returns the remainder of a division — same as the % operator.
POWER(num, exponent)	Raises a number to the given power (exponent).

```
SELECT TRUNCATE(789.8834, 2);
-- Output: 789.88  (note: it cuts off, it does not round)
```

Example: "Display names of employees who have an even number of characters in their name."

```
SELECT ename FROM emp
WHERE MOD(LENGTH(ename), 2) = 0;
```

13.4 Date & Time Functions

Function	Description
CURDATE()	Returns the current date, according to the system.
NOW()	Returns the current date and time, according to the system.

Function	Description
MONTH(date)	Extracts the month from a given date.
YEAR(date)	Extracts the year from a given date.
DAYNAME(date)	Returns the name of the weekday for a given date (e.g. 'Monday').
LAST_DAY(date)	Returns the last date of the month for a given date.

13.5 Conditional Functions

IF()

IF() evaluates a condition and returns one of two values, depending on whether the condition is true or false.

```
IF(expression, true_value, false_value)
```

Example: "Display names of employees, along with their salary marked as 'HIGH' if it is greater than 3000, otherwise 'LOW'."

```
SELECT ename, sal,
       IF(sal > 3000, 'HIGH', 'LOW') AS sal_category
FROM emp;
```

CASE

CASE works like an IF-ELSE ladder — it evaluates a list of conditions in order and returns the result for the first matching condition. If none of the conditions match, it returns the ELSE value (if provided).

```
CASE
    WHEN cond1 THEN result1
    WHEN cond2 THEN result2
    ...
    ELSE default_result
END
```

Example: "Display name and salary — mark it 'HIGH' if salary > 3000, 'MEDIUM' if between 1100 and 2000, otherwise 'LOW'."

```
SELECT ename, sal,
  CASE
      WHEN sal > 3000 THEN 'HIGH'
      WHEN sal BETWEEN 1100 AND 2000 THEN 'MEDIUM'
      ELSE 'LOW'
  END AS sal_category
FROM emp;
```

Example: "Display employee name, department name, and mark the department as 'MONEY' if it is Accounts or Sales, and 'OTHER' otherwise."

```
SELECT ename, dname,
  CASE
      WHEN dname IN ('ACCOUNTS', 'SALES') THEN 'MONEY'
      ELSE 'OTHER'
  END AS dept_category
FROM emp INNER JOIN dept ON emp.deptno = dept.deptno;
```

13.6 Null-Handling Functions

Function	Description
ISNULL(expr)	Returns 1 if the given expression is NULL, and 0 if it is not NULL.
IFNULL(expr, replacement)	Returns the replacement value if the expression is NULL; otherwise returns the expression itself.
COALESCE(expr1, expr2, ...)	Evaluates each expression in order and returns the first one that is not NULL. If every expression is NULL, it returns NULL as well.

Example: "Display name and commission of employees — if commission is NULL, show it as 0."

```
SELECT ename, IFNULL(comm, 0) AS comm
FROM emp;
```

Example: "Display department name with total commission greater than 500, treating NULL commissions as 0."

```
SELECT dname, SUM(COALESCE(comm, 0)) AS total_comm
FROM emp INNER JOIN dept ON emp.deptno = dept.deptno
GROUP BY dname
HAVING SUM(COALESCE(comm, 0)) > 500;
```

Example: "Display manager name and number of employees reporting to each manager — if an employee has no manager, label it 'No Manager'."

```
SELECT COALESCE(m.ename, 'No Manager') AS manager, COUNT(e.empno)
FROM emp e INNER JOIN emp m ON e.mgr = m.empno
GROUP BY COALESCE(m.ename, 'No Manager');
```

13.7 Aggregate (Multi-Row / Group) Functions

These functions work on a group of rows and return a single result. They work only on numerical data, except COUNT, which can be used on any data type.

Function	Description
MAX()	Returns the highest value in a column.
MIN()	Returns the lowest value in a column.
SUM()	Returns the total sum of values in a column.
AVG()	Returns the average of values in a column.
COUNT()	Returns the number of rows.

- COUNT(column_name) — ignores NULL values in that column.
- COUNT(*) — counts all rows, including those with NULLs.

Example: "Display the most recent hire date from the emp table."

```
SELECT MAX(hiredate) FROM emp;
```

14. Important Clauses

14.1 GROUP BY

GROUP BY is used to group similar records together, typically so an aggregate function can be applied to each group. It works row-by-row and, if any other clause executes after it, that clause runs group-by-group.

Example: "Display the sum of salary for each job."

```
SELECT job, SUM(sal) FROM emp GROUP BY job;
```

14.2 HAVING

HAVING is used to filter records after grouping — it filters groups, whereas WHERE filters individual rows before grouping.

Example: "Display duplicate salaries from the employee table."

```
SELECT sal FROM emp
GROUP BY sal
HAVING COUNT(*) > 1;
```

14.3 ORDER BY

ORDER BY is used to arrange the records in a particular order — ascending or descending. By default, sorting is in ascending order. ASC = ascending, DESC = descending. When multiple columns are given, priority is given to the first field listed.

```
SELECT job, sal FROM emp ORDER BY job, sal;
SELECT DISTINCT sal FROM emp ORDER BY sal;
```

14.4 LIMIT

LIMIT is a clause used to restrict the number of records returned by a query. It should always be placed at the end of the query.

```
-- Without OFFSET
SELECT col_name FROM table_name LIMIT row_count;

-- With OFFSET (skips a number of records first)
SELECT col_name FROM table_name LIMIT offset, row_count;
```

Example: "Get the 2 earliest-hired employees."

```
SELECT * FROM emp
```

```
ORDER BY hiredate ASC
LIMIT 2;
```

15. Subqueries

A subquery is a query written inside another query (also called an inner query). It is often used when a condition depends on a value that must first be looked up from the database itself.

15.1 Types of Subqueries

Type	Description
Single-Row Subquery	Returns exactly one value as its output. Can be used with =, >, <, etc.
Multi-Row Subquery	Returns more than one row/value as output. Must be used with IN, ANY, ALL, EXISTS.

15.2 Employee–Manager Relationships

A very common real-world subquery pattern involves employee-manager relationships, typically requiring two steps:

- Step 1: Identify the manager (or the relevant employee) using an inner query.
- Step 2: Use that result in the outer query to find the related employees.

Example: "Display the manager's name of FORD."

```
SELECT ename FROM emp
WHERE empno = (
    SELECT mgr FROM emp WHERE ename = 'FORD'
);
```

Example: "Display the maximum salary among employees who report to KING."

```
SELECT MAX(sal) FROM emp
WHERE mgr = (
    SELECT empno FROM emp WHERE ename = 'KING'
);
```

15.3 Subqueries Returning Multiple Rows — Using IN

When a subquery can return more than one value, the equality operator (=) cannot be used — IN must be used instead.

Example: "Display the name and location of the department where employees Allen and Jones work."

```
SELECT dname, loc FROM dept
WHERE deptno = (
    SELECT deptno FROM emp WHERE ename IN ('ALLEN', 'JONES')
);
```

```
-- Note: use IN here, NOT '=', since the subquery
-- can return more than one row.
```

Example: "Display the number of employees working at the location 'CHICAGO'."

```
SELECT COUNT(*) FROM emp
WHERE deptno = (
    SELECT deptno FROM dept WHERE loc = 'CHICAGO'
);
```

15.4 More Subquery Examples

Example: "Display details of employees whose salary is greater than Allen's salary."

```
SELECT * FROM emp
WHERE sal > (
    SELECT sal FROM emp WHERE ename = 'ALLEN'
);
```

Example: "Display name and job of employees hired after James, who must also work in the same department as Miller."

```
SELECT ename, job FROM emp
WHERE hiredate > (
    SELECT hiredate FROM emp WHERE ename = 'JAMES'
)
AND deptno = (
    SELECT deptno FROM emp WHERE ename = 'MILLER'
);
```

15.5 Correlated Subqueries

A correlated subquery is one where the inner query and the outer query are inter-related — the inner query refers to a column from the outer query, so it must be re-evaluated once for every row processed by the outer query (unlike a normal subquery, which runs only once).

EXISTS

EXISTS is a unary subquery operator. It returns TRUE if the subquery returns at least one row (no matter what the actual values are), and FALSE if the subquery returns no rows at all.

> ***Correction from notes:*** *The EXISTS operator checks only whether any rows are returned, not whether a value is NULL. It evaluates to TRUE when the subquery's result set is non-empty, and FALSE when it is empty.*

NOT EXISTS

NOT EXISTS is also a unary subquery operator — the exact opposite of EXISTS. It returns TRUE when the subquery returns no rows (an empty result), and FALSE when the subquery returns at least one row.

Example: "Display names of employees along with the names of their respective managers, using a correlated join."

```
SELECT e.ename, m.ename AS manager_name
FROM emp e INNER JOIN emp m
ON e.mgr = m.empno;
```

15.6 Finding the Nth Highest Salary

A classic correlated-subquery pattern is finding the Nth highest value in a column — for example, the Nth highest salary. The idea: for each candidate salary, count how many distinct salaries are strictly greater than it. If exactly (N-1) salaries are greater, the candidate is the Nth highest.

```
-- Syntax to find the Nth maximum salary
SELECT e1.sal FROM emp e1
WHERE (
    SELECT COUNT(DISTINCT e2.sal)
    FROM emp e2
    WHERE e1.sal < e2.sal
) = N-1;

-- Example: 2nd highest salary (N = 2)
SELECT e1.sal FROM emp e1
WHERE (
    SELECT COUNT(DISTINCT e2.sal)
    FROM emp e2
    WHERE e1.sal < e2.sal
) = 1;
```

> ***Note:*** *This query works because it is correlated — for every row e1, the inner query re-runs, comparing e1's salary against every other row e2. This is a very common interview question for SQL beginners.*

Example: "Display employees who earn a salary greater than the average salary of their own location."

```
SELECT * FROM emp e
WHERE sal > (
    SELECT AVG(sal) FROM emp e2
    INNER JOIN dept d ON e2.deptno = d.deptno
    WHERE d.loc = (
        SELECT loc FROM dept WHERE deptno = e.deptno
    )
);
```

16. Functional Dependency

Functional Dependency is used to define the relationship between the attributes of a table. If we have a relation R(x, y, z), a functional dependency x -> y means "x determines y" — in other words, for every value of x, there is exactly one corresponding value of y.

- x is called the Determinant.
- y is called the Dependent Attribute.

16.1 Types of Functional Dependency

Type	Description
Total (Full) Functional Dependency	All the non-key attributes are dependent on the entire (single) key attribute.
Partial Functional Dependency	Occurs only when the key is composite — a non-key attribute depends on only part of the composite key, not the whole key.
Transitive Functional Dependency	One non-key attribute determines another non-key attribute, which in turn is dependent on the key attribute (an indirect dependency, X -> Y -> Z, giving X -> Z indirectly).

Total Functional Dependency — Example

emp(empno, ename, sal, comm, hiredate, job, deptno) — here every non-key attribute depends fully on the single key attribute empno.

Partial Functional Dependency — Example

Consider a relation R(A, B, C) where (A, C) together form the composite key, and A alone is enough to determine B:

```
R(A, B, C)
(A, C) -> Composite Key
(A, C) -> B     -- full dependency on the composite key
A -> B          -- but A alone already determines B
                -- => Partial Functional Dependency

Example: R(father, mother, son, daughter)
(father, mother) -> Composite key attribute
father -> daughter   -- a partial dependency (mother isn't needed)
```

Transitive Functional Dependency — Example

```
R(X, Y, Z)
X is the Key attribute
Y, Z are Non-key attributes
```

```
X -> Y  and  Y -> Z
=> X -> Z   (indirectly, through Y) -- Transitive Dependency

Example: R(father, mother, son, daughter)
father -> Key attribute
father -> daughter
daughter -> son
=> Indirectly, father -> son   (Transitive Dependency)
```

17. Normalization

Normalization is the process of dividing a large table into smaller tables, in order to remove data redundancy and anomalies. A table is said to be "normalized" once it no longer has unnecessary data redundancy or update/insert/delete anomalies.

17.1 Why Normalization?

- Removes data redundancy — saves storage space.
- Improves data integrity — no conflicting copies of the same data.
- Gives the database a better structure — easier to maintain.

17.2 Levels (Normal Forms) of Normalization

Normal Form	Rule
1NF	The table must not have multi-valued data, and must not have duplicate rows/columns — every value must be atomic (indivisible).
2NF	The table must already be in 1NF, and must not have any Partial Functional Dependency.
3NF	The table must already be in 2NF, and must not have any Transitive Functional Dependency.
BCNF (Boyce-Codd NF)	The table must already be in 3NF, and every determinant in the table must be a candidate key. It is also called 3.5NF, and is a stricter version of 3NF.

17.3 Worked Example

Start with the following unnormalized table, which has a multi-valued "skill" column:

empid	ename	dname	loc	sal	skill
1	Amit	IT	Mumbai	50000	SQL, Java
2	Priya	IT	Mumbai	45000	Java, Python, SQL

empid	ename	dname	loc	sal	skill
3	Raj	HR	Delhi	60000	Excel

Step 1 — Convert to 1NF

The skill column has multiple values in a single cell, which violates 1NF. We split it out into its own table, one skill per row:

empid	ename	dname	loc	sal
1	Amit	IT	Mumbai	50000
2	Priya	IT	Mumbai	45000
3	Raj	HR	Delhi	60000

empid	skill
1	SQL
1	Java
2	Java
2	Python
2	SQL
3	Excel

Step 2 — Convert to 2NF

To be in 2NF, the table must already be in 1NF, and must have no partial dependency. This rule only matters when a table has a composite primary key — a non-key column must depend on the entire composite key, not just part of it. In our employee table, empid alone is the key (not composite), so this table already satisfies 2NF.

Step 3 — Convert to 3NF

To be in 3NF, the table must already be in 2NF and must have no transitive dependency. In our example: empid -> dname, and dname -> loc. This is a transitive dependency (empid determines loc only indirectly, through dname), so we split department details into their own table:

```
emp(empid, ename, sal, deptid)
dept(deptid, dname, loc)
```

> ***Note:*** *Notice the department's name and location no longer repeat for every employee — they are stored once per department and simply referenced using deptid (a foreign key). This is exactly what normalization achieves: less redundancy, and a single reliable place to update department details.*

17.4 Who Performs Normalization?

- Database Administrator (DBA)
- Database Designer
- Developer

18. Window Functions

A window function performs a calculation across a set of rows that are related to the current row — without collapsing those rows into a single output row the way GROUP BY does. Instead, the calculated value is shown alongside every individual row.

18.1 OVER()

OVER() tells SQL not to collapse the rows — instead, it calculates the function across a defined "window" of rows and displays the result next to each individual row.

18.2 PARTITION BY

PARTITION BY divides the rows into groups (or "windows") — it is used to break the data into groups, similar in spirit to GROUP BY, but without merging the rows in the final output.

Example: "Show the total salary of each department alongside every individual employee row."

```
SELECT deptno, ename, sal,
       SUM(sal) OVER (PARTITION BY deptno) AS total_sal
FROM emp;
```

> ***Note:*** *Here, each row still shows the individual employee's salary, but total_sal repeats the total for that employee's department across every one of its rows — the rows are not merged.*

18.3 ORDER BY within a Window

ORDER BY (used inside the OVER() clause) defines the sequence of rows inside each partition/group — this is essential for running totals and ranking functions. It controls the order in which the calculation proceeds inside each group.

```
SELECT ename, deptno, sal,
       SUM(sal) OVER (PARTITION BY deptno ORDER BY sal) AS running_total
FROM emp;
```

18.4 Types of Window Functions

Category	Functions
Ranking Functions	RANK(), DENSE_RANK(), ROW_NUMBER()
Aggregate Functions	SUM(), MIN(), MAX(), AVG(), COUNT() — used with OVER()
Value / Offset Functions	LEAD(), LAG()
Terminal (Frame-Boundary) Functions	FIRST_VALUE(), LAST_VALUE(), NTH_VALUE()

Category	Functions
Distribution Functions	NTILE(), CUME_DIST(), PERCENT_RANK()

> ***Note:*** *MySQL itself does not use the exact label "terminal function" — it simply lists all of these under Window Functions. But FIRST_VALUE(), LAST_VALUE(), and NTH_VALUE() are commonly described this way because they specifically return the value sitting at the terminal points (the start, the end, or a chosen position) of a window frame, rather than ranking rows or offsetting to a neighboring row.*

RANK()

RANK() gives each row a rank number, but if two or more rows tie for the same rank, it skips the following rank number(s) — e.g. two rows tied for rank 1 means the next rank given out is 3, not 2.

```
SELECT ename, sal,
       RANK() OVER (ORDER BY sal DESC) AS sal_rank
FROM emp;
```

DENSE_RANK()

DENSE_RANK() works just like RANK(), except it does not skip any rank numbers after a tie — the next rank after a tie is always the very next integer.

```
SELECT ename, sal,
       DENSE_RANK() OVER (ORDER BY sal DESC) AS sal_dense_rank
FROM emp;
```

ROW_NUMBER()

ROW_NUMBER() assigns a unique, sequential number to every row within its partition, regardless of whether values are tied — no two rows ever get the same number.

```
SELECT ename, sal,
       ROW_NUMBER() OVER (ORDER BY sal DESC) AS row_num
FROM emp;
```

> ***Note:*** *Difference at a glance: for tied values, RANK() leaves a gap in the numbering, DENSE_RANK() leaves no gap, and ROW_NUMBER() ignores ties completely and just keeps counting 1, 2, 3, 4...*

LEAD() and LAG()

LEAD() and LAG() let you look at a value from another row, relative to the current row, without needing a self-join. LAG() looks backward (at a previous row); LEAD() looks forward (at a following row).

```
-- Compare each employee's salary with the next employee's salary
SELECT ename, sal,
       LEAD(sal) OVER (ORDER BY sal DESC) AS next_lower_sal,
       LAG(sal)  OVER (ORDER BY sal DESC) AS next_higher_sal
```

```
FROM emp;
```

18.5 Terminal (Frame-Boundary) Functions

These functions return the value sitting at a specific boundary position — the first row, the last row, or the Nth row — of the current window frame, rather than a neighboring row like LEAD/LAG do.

FIRST_VALUE()

Returns the value of the given expression from the first row of the window frame.

```
SELECT ename, deptno, sal,
       FIRST_VALUE(ename) OVER (
           PARTITION BY deptno ORDER BY sal DESC
       ) AS top_earner_in_dept
FROM emp;
```

LAST_VALUE()

Returns the value of the given expression from the last row of the window frame. Note: by default, the frame only extends up to the current row, so LAST_VALUE() often needs an explicit frame (e.g. ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) to actually reach the true last row of the partition.

```
SELECT ename, deptno, sal,
       LAST_VALUE(ename) OVER (
           PARTITION BY deptno ORDER BY sal DESC
           ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
       ) AS lowest_earner_in_dept
FROM emp;
```

NTH_VALUE()

Returns the value of the given expression from the Nth row of the window frame. If the frame does not contain that many rows, it returns NULL.

```
SELECT ename, deptno, sal,
       NTH_VALUE(ename, 2) OVER (
           PARTITION BY deptno ORDER BY sal DESC
           ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
       ) AS second_highest_earner
FROM emp;
```

18.6 Distribution Functions (Bonus)

A few more window functions that are useful once RANK/DENSE_RANK/ROW_NUMBER feel comfortable:

Function	Description
NTILE(n)	Divides the rows in a partition into n roughly equal-sized buckets, and returns the bucket number for each row. Useful for splitting data into quartiles, deciles, etc.
CUME_DIST()	Returns the cumulative distribution of the current row — the fraction of rows in the partition with a value less than or equal to the current row's value (range: 0 to 1).
PERCENT_RANK()	Returns the relative rank of the current row as a percentage, calculated as (rank - 1) / (total rows - 1) (range: 0 to 1).

Example: "Split employees into 4 salary quartiles."

```
SELECT ename, sal,
       NTILE(4) OVER (ORDER BY sal DESC) AS salary_quartile
FROM emp;
```

19. Sample Practice Schema

Many of the example queries above use a classic HR-style schema (similar to the famous Oracle 'Scott/EMP-DEPT' schema). A slightly extended version could look like this:

```
employee(empid, empname, job, sal, hiredate, deptid, mgrid)
department(deptid, deptname, loc, mgrid)
sal_history(historyid, empid, oldsal, newsal, chgdate)
project(projid, projname, budget)
emp_project(empid, projectid, role)
```

Using tables like these, you can practice everything covered in this document — joins, subqueries, window functions, aggregate functions, grouping, filtering, keys, and normalization — on realistic, connected data.

20. Quick Reference Summary

20.1 SQL Statement Categories

Category	Key Commands
DDL	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	INSERT, UPDATE, DELETE
TCL	COMMIT, ROLLBACK, SAVEPOINT
DCL	GRANT, REVOKE
DQL	SELECT

20.2 Join Types at a Glance

Join	Returns
CROSS JOIN	Every row of table1 combined with every row of table2 (Cartesian product)
INNER JOIN	Only the matching rows between both tables
NATURAL JOIN	Inner-join-like result on matching column names; cross join if none match
LEFT OUTER JOIN	All rows from the left table + matching rows from the right
RIGHT OUTER JOIN	All rows from the right table + matching rows from the left
FULL OUTER JOIN	All rows from both tables, matched where possible
SELF JOIN	A table joined with itself (e.g. employee-manager)

20.3 Logical Order to Learn / Write SQL Queries

- 1. SELECT and FROM — pick columns and a table
- 2. JOIN — bring in related tables, if needed
- 3. WHERE — filter rows
- 4. GROUP BY — group rows
- 5. HAVING — filter groups
- 6. ORDER BY — sort the result
- 7. LIMIT — restrict the number of rows returned
- 8. Subqueries / Window Functions — for advanced, layered logic

— End of Notes —